LECTURE SERIES · DATABASE SYSTEMS

Relational Algebra

The Formal Query Language Behind SQL

Md. Rakib Trofder

Lecturer, Bangladesh Army University of Science & Technology

Basics · Selection & Projection · Set Operations · Joins

A PROCEDURAL QUERY LANGUAGE

What is Relational Algebra?

Relational algebra is a **procedural** query language: every operation takes one or two relations and returns a new relation. Because the result is always a relation, operations **chain together** — the closure property. SQL is built on top of it.

Unary Operations

operate on one relation

- σ **Selection**
- π **Projection**
- ρ **Rename**

Set Operations

need union-compatibility

- ∪ **Union**
- ∩ **Intersection**
- − **Difference**

Binary Operations

combine two relations

- × **Cartesian Product**
- ⋈ **Join**

THE FUNDAMENTAL UNARY OPERATORS

Selection & Projection

Selection (σ)

Keeps the ROWS that satisfy a condition — filters horizontally.

selection

$$\sigma_{dept='CSE'}(Students)$$

→ only the CSE students

Projection (π)

Keeps the chosen COLUMNS, dropping duplicates — filters vertically.

projection

$$\pi_{name,\ cgpa}(Students)$$

→ just the name & cgpa columns

Operations compose — nest them freely:

$$\pi_{name}(\sigma_{cgpa>3.5}(Students))$$

SETTING THE STAGE FOR JOINS

Rename & Cartesian Product

Rename (ρ)

Renames a relation and/or its attributes — essential for self-joins and clarity.

rename

$$\rho_{S(sid,\ sname)}(Students)$$

Cartesian Product (×)

Pairs every row of R with every row of S — the raw combination of two relations.

product

$$Students \times Courses$$

Why it matters: the result has degree = deg(R) + deg(S) and $|R| \times |S|$ rows — apply a selection to it and you get a JOIN.

COMBINING TWO RELATIONS

Set Operations

Set operations treat relations as sets of rows. They require **union-compatible** relations — the same number of attributes, with matching domains.



Union

All rows appearing in R or S (duplicates removed).

$$R \cup S$$



Intersection

Rows that appear in both R and S.

$$R \cap S$$



Difference

Rows in R that are not in S.

$$R - S$$

COMBINING RELATED RELATIONS

Joins

A join pairs rows from two relations that satisfy a condition — conceptually a Cartesian product followed by a selection.

Theta Join

Pairs rows meeting any condition θ (<, >, ≠ …).

Equi Join

A theta join whose condition is equality (=).

Natural Join

Auto-matches on common attributes; keeps one copy of each.

NATURAL JOIN EXAMPLE

Students ⋈ Enrolls

Students

SID	Name
1	Rakib
2	Alifa

Enrolls

SID	CID
1	C1
2	C2

= *matched on the shared SID* ↓

SID	Name	CID
1	Rakib	C1
2	Alifa	C2

KEEPING THE UNMATCHED ROWS

Outer Joins

An inner join drops rows with no match. Outer joins keep those rows too, filling the missing attributes with NULL.

Left Outer

Every row of the LEFT relation is kept; unmatched right side becomes NULL.

Right Outer

Every row of the RIGHT relation is kept; unmatched left side becomes NULL.

Full Outer

Unmatched rows from BOTH relations are kept, padded with NULL.

LEFT OUTER JOIN — Nadia has no enrollment, so CID is NULL

SID	Name	CID
1	Rakib	C1
2	Alifa	C2
3	Nadia	*NULL*

Remember: NULL marks “no matching row”, not zero or empty — it simply means the value is unknown here.

KEY TAKEAWAYS

Recap

Relational Algebra

Procedural — relations in, a relation out

Selection & Projection

σ picks rows, π picks columns

Rename & Product

ρ and × set up joins

Set Operations

Union, Intersection, Difference

Joins

Theta, equi & natural (⋈)

Outer Joins

Keep unmatched rows with NULL

Thank You

Md. Rakib Trofder
Lecturer, Bangladesh Army University of Science & Technology